FACULTY OF COMPUTING

SLIIT Kandy Uni | 2Y1S | Group ID : 2026-Y2-S1-KU-24 (Group 24)

IT2140 — Database Design and Development

STREETIFY

Smart Web based Transportation Platform

Assignment Part 02

Relational Schema Mapping | DDL | DML | Queries | Stored Procedure | Trigger

---

Index No.	Name	Module
IT25102240	Rammandalagedara R.V.S. (Vidura)	Super Admin & Governance
IT25102208	Nayanamina A.R.L.P. (Lahiru)	User Account & Verification
IT25102207	Dharmakeerthi W.A.C.B. (Chanuka)	Ride Booking & Dispatch Engine
IT25102241	Senaka K.A.T. (Tharindu)	Trip Progress & Driver Telemetry
IT25102225	Edirisinghe E.A.R.N.D. (Daham)	Payment & Ledger Management
IT25102193	Weerasingha W.A.M.B. (Mithun)	Review & Dispute Management

Submission: 27 th September 2026

Contents

Part A: Relational Schema Mapping and Refinement 3

A.1 Relational Schema Overview 3

A.2 EER-to-Relational Mapping Rules Applied 5

A.3 Schema Design Decisions & Viva Justifications 6

A.4 Normalization Verification (1NF, 2NF, 3NF, BCNF) 7

Part B: SQL DDL Implementation 8

B.1 Full DDL Script — streetify_db 8

B.2 Constraint & Index Summary 13

Part C: Insert Sample Data 14

C.1 Master Seed Script 14

C.2 Verification Queries (SELECT * from each table) 18

D.1 Simple SELECT — Driver Fleet Operations Dashboard 20

D.2 JOIN — Passenger Trip History with Driver Details 21

D.3 Aggregation — Revenue Summary by Payment Method 22

D.4 GROUP BY / HAVING — High-Performing Driver Identification 23

D.5 Subquery — Above-Average Fare Trips (Surge Pricing Analysis) 24

D.6 Additional Team Module Queries (03_team_member_queries.sql) 25

Part E: Stored Procedure — usp_ProcessTripPayment 30

E.1 Business Logic & Design Rationale 30

E.2 Complete Stored Procedure Code 31

E.3 Demonstration Script 34

Part F: Trigger — trg_ProtectFinalizedTrips 36

F.1 Trigger Design Rationale 36

F.2 Complete Trigger Code 37

F.3 Demonstration Script 39

References 40

Part A: Relational Schema Mapping and Refinement

A.1 Relational Schema Overview

The Streetify relational schema spans 9 tables across 6 functional modules.

Notation: PK=Primary Key, FK=Foreign Key, UQ=Unique, NN=Not Null.

Module	Relation Name	Attributes & Key Constraints
Users	users	id PK IDENTITY, email UQ NN, password_hash NN, first_name NN, last_name NN, phone, dtype NN, role NN, admin_role, active BIT NN, suspended BIT NN, suspension_reason, suspended_until, wallet_balance DECIMAL NN, preferred_payment_method, license_number, nic, verification_status, average_rating, total_trips INT, commission_debt DECIMAL, is_online BIT, current_lat FLOAT, current_lng FLOAT, created_at, updated_at
Fleet	vehicles	id PK IDENTITY, driver_id FK->users.id UQ NN, vehicle_type NN, number_plate UQ NN, year_of_manufacture NN, make, model, color, created_at
Fleet	driver_documents	id PK IDENTITY, driver_id FK->users.id NN, doc_type NN, original_filename NN, file_path NN, file_size_bytes, content_type, status NN DEFAULT PENDING, reviewer_note, uploaded_at, reviewed_at
Trips	trips	id PK IDENTITY, passenger_id FK->users.id NN, driver_id FK->users.id, pickup_address NN, pickup_lat NN, pickup_lng NN, dropoff_address NN, dropoff_lat NN, dropoff_lng NN, distance_km, ride_type NN, base_fare, per_km_rate,

		platform_fee, total_fare, platform_commission, driver_net, status NN, accepted_at, arrived_at, started_at, completed_at, cancelled_at, cancellation_reason, no_show_fee, payment_method, is_paid BIT NN, created_at, updated_at
Billing	payments	id PK IDENTITY, trip_id FK->trips.id UQ NN, passenger_id FK->users.id NN, driver_id FK->users.id NN, gross_amount NN, platform_commission NN, driver_net NN, payment_method NN, status NN, retry_count INT NN, failure_reason, transaction_ref, created_at, processed_at
Engage	reviews	id PK IDENTITY, trip_id FK->trips.id UQ NN, passenger_id FK->users.id NN, driver_id FK->users.id NN, rating INT CHECK(1-5) NN, comment, created_at
Support	dispute_tickets	id PK IDENTITY, passenger_id FK->users.id NN, trip_id FK->trips.id, subject NN, description NN, dispute_type NN, requested_refund_amount, status NN, resolution_note, approved_refund_amount, resolved_by_staff_id FK->users.id, created_at, updated_at, resolved_at
Audit	audit_logs	id PK IDENTITY, performed_by_staff_id NN, performed_by_email NN, action_type NN, description NN, target_user_id, target_entity_type, target_entity_id, created_at

A.2 EER-to-Relational Mapping Rules Applied

The following 8 standard mapping rules were applied to convert the Streetify EER diagram into the relational schema:

Rule 1 — Regular (Strong) Entity Types

Each strong entity in the EER (USER, TRIP, PAYMENT, VEHICLE, REVIEW, DISPUTE_TICKET, AUDIT_LOG) maps to its own relation. All simple attributes become columns; the EER's primary key attribute becomes the relational PK. Result: 8 base relations.

Rule 2 — Composite Attributes

Composite attributes PickupLocation and DropoffLocation from the TRIP entity are flattened into atomic columns (pickup_lat, pickup_lng, pickup_address; dropoff_lat, dropoff_lng, dropoff_address) inside the trips table. This maintains 1NF and avoids extra joins.

Rule 3 — Multivalued Attributes

The EER ContactNo multivalued attribute on USER is simplified to a single phone column (one primary contact per user). For multi-contact support, a separate user_contacts table would be added — this design decision is documented in A.3.

Rule 4 — Weak Entity Types

AUDIT_LOG is a weak entity, identified by the staff member who performed the action. A surrogate IDENTITY PK is added in addition to the identifying FK (performed_by_staff_id) per MSSQL best practice for index performance.

Rule 5 — 1:1 Relationships

The 1:1 relationships TRIP-PAYMENT and TRIP-REVIEW are enforced by placing trip_id as a UNIQUE FOREIGN KEY in both payments and reviews. The UNIQUE constraint guarantees exactly one payment and one review per completed trip at the database level.

Rule 6 — 1:N Relationships

All one-to-many relationships (DRIVER->TRIPS, PASSENGER->TRIPS, DRIVER->VEHICLES, DRIVER->DRIVER_DOCUMENTS) are represented by placing the PK of the 'one' side as an FK in the 'many' side table. E.g., driver_id in trips references users.id.

Rule 7 — ISA / Specialization (USER Hierarchy)

The EER defines a Total, Disjoint specialization: USER -> {PASSENGER, DRIVER, ADMIN}. We implemented Option D — Single Table Inheritance (STI) — using the dtype discriminator column and role column inside the unified users table. This minimizes JOIN overhead for authentication, the highest-frequency query in the platform.

Rule 8 — Indexes on FK Columns

All FK columns used in WHERE clauses or JOIN conditions are backed by explicit CREATE INDEX statements (idx_trips_passenger, idx_trips_driver, idx_trips_status, etc.) for query performance at scale. This is a production best practice not captured in the basic mapping rules.

A.3 Schema Design Decisions & Viva Justifications

*** STI vs. Table-per-Subtype for USER Hierarchy**

We chose Single Table Inheritance (STI) because authentication — the highest-frequency database operation in Streetify — requires fetching role, active, and suspended in a single row. With separate subtype tables (passengers, drivers), every login would require a JOIN. STI reduces this to a single indexed scan on users.email. The trade-off (NULL columns for role-specific attributes) is acceptable because the application enforces that dtype='PASSENGER' rows will never populate driver-specific columns.

*** UNIQUE Constraint on payments.trip_id for 1:1 Enforcement**

Rather than relying on application-layer logic, the 1:1 relationship between TRIP and PAYMENT is enforced at the database level with a UNIQUE constraint on trip_id. This provides idempotency guarantees regardless of network retries or concurrent API requests.

*** BIGINT IDENTITY(1,1) Surrogate Keys**

All tables use BIGINT IDENTITY surrogate PKs. While natural keys (email, number_plate) are constrained UNIQUE as candidate keys, the integer surrogate enables optimal B-tree index performance, compact FK references, and compatibility with future table partitioning.

*** DECIMAL(10,2) for Monetary Columns**

All fare, commission, and balance columns use DECIMAL(10,2). IEEE 754 FLOAT introduces binary rounding errors (0.1 + 0.2 != 0.3) that are unacceptable in financial systems. DECIMAL is an exact numeric type, mandatory for any money calculation.

*** ON DELETE CASCADE for vehicles and driver_documents**

These tables use CASCADE because a vehicle and its documents have no existence independent of their driver — this is an existential dependency. Core financial tables (trips, payments) deliberately omit CASCADE to preserve transaction history even if a user account is deactivated.

*** Storing platform_commission and driver_net as Snapshots**

Although derivable (15% of gross), these values are stored at payment time as immutable financial snapshots. If the commission rate changes in future, historical payment records remain accurate and auditable without recalculation.

A.4 Normalization Verification (1NF, 2NF, 3NF, BCNF)

All 9 Streetify relations have been verified against the criteria for Third Normal Form and Boyce-Codd Normal Form:

Normal Form	Rule	Streetify Compliance Evidence	Status
1NF	Atomic Values	All columns are atomic — no arrays, no repeating groups, no composite cell values. Composite EER attributes (PickupLocation, DropoffLocation) are flattened. phone stores one primary contact number.	PASS — All 9 Tables
2NF	No Partial Dependencies	All non-key attributes are fully dependent on the ENTIRE primary key. Since all tables use single-column surrogate PKs (IDENTITY), partial dependencies are architecturally impossible.	PASS — All 9 Tables
3NF	No Transitive Dependencies	No non-key attribute determines another non-key attribute. platform_commission in payments is stored as a snapshot value, not derived from another non-key column at query time. rating in reviews does not determine any other attribute.	PASS — All 9 Tables
BCNF	Every Determinant is a Superkey	For every FD X->Y, X is a superkey. All determinants that are not the PK (email, number_plate, trip_id in payments) are declared UNIQUE constraints, making them candidate keys — satisfying BCNF.	PASS — All 9 Tables

Part B: SQL DDL Implementation

B.1 Full DDL Script — streetify_db

The script creates the database, drops existing tables in FK-safe order, creates all 9 tables with full constraints, and adds performance indexes.

Step 1 — Database Creation & Clean Drop

```
USE streetify_db;
GO

-- Drop in FK-safe dependency order
IF OBJECT_ID(N'audit_logs',       N'U') IS NOT NULL DROP TABLE audit_logs;
IF OBJECT_ID(N'dispute_tickets',  N'U') IS NOT NULL DROP TABLE dispute_tickets;
IF OBJECT_ID(N'reviews',          N'U') IS NOT NULL DROP TABLE reviews;
IF OBJECT_ID(N'payments',         N'U') IS NOT NULL DROP TABLE payments;
IF OBJECT_ID(N'trips',            N'U') IS NOT NULL DROP TABLE trips;
IF OBJECT_ID(N'driver_documents', N'U') IS NOT NULL DROP TABLE driver_documents;
IF OBJECT_ID(N'vehicles',         N'U') IS NOT NULL DROP TABLE vehicles;
IF OBJECT_ID(N'users',            N'U') IS NOT NULL DROP TABLE users;
GO
```

Table: users

```
-- TABLE: users  (STI – Passengers, Drivers, and Admins in one table)
CREATE TABLE users (
    id                      BIGINT IDENTITY(1,1) PRIMARY KEY,
    dtype                   VARCHAR(31)    NOT NULL,          -- USER|PASSENGER|DRIVER
    first_name              NVARCHAR(100)  NOT NULL,
    last_name               NVARCHAR(100)  NOT NULL,
    email                   NVARCHAR(150)  NOT NULL UNIQUE,
    password_hash           NVARCHAR(255)  NOT NULL,
    phone                   NVARCHAR(20)   NULL,
    role                    VARCHAR(20)    NOT NULL,          -- PASSENGER|DRIVER|ADMIN
    admin_role              VARCHAR(30)    NULL,              -- SUPER_ADMIN|USER_MGMT|...
    active                  BIT            NOT NULL DEFAULT 1,
    suspended               BIT            NOT NULL DEFAULT 0,
    suspension_reason       NVARCHAR(500)  NULL,
    suspended_until         DATETIME2      NULL,
    wallet_balance          DECIMAL(10,2)  NOT NULL DEFAULT 0.00,
    -- Passenger-specific
    preferred_payment_method  VARCHAR(20)  NULL,              -- CASH|CARD|WALLET
    -- Driver-specific
    license_number          NVARCHAR(50)   NULL,
    nic                     NVARCHAR(20)   NULL,
    verification_status     VARCHAR(30)    NULL DEFAULT 'PENDING_VERIFICATION',
    average_rating          FLOAT          NULL DEFAULT 5.0,
    total_trips             INT            NULL DEFAULT 0,
    commission_debt         DECIMAL(10,2)  NULL DEFAULT 0.00,
    is_online               BIT            NULL DEFAULT 0,
    current_lat             FLOAT          NULL,
    current_lng             FLOAT          NULL,
    created_at              DATETIME2      NOT NULL DEFAULT GETDATE(),
    updated_at              DATETIME2      NOT NULL DEFAULT GETDATE()
);
CREATE INDEX idx_users_email  ON users(email);
CREATE INDEX idx_users_role   ON users(role);
CREATE INDEX idx_users_dtype  ON users(dtype);
GO
```

Tables: vehicles & driver_documents

```
-- TABLE: vehicles  (1:1 with driver, CASCADE on driver delete)
CREATE TABLE vehicles (
    id                  BIGINT IDENTITY(1,1) PRIMARY KEY,
    driver_id           BIGINT         NOT NULL UNIQUE
                        FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE,
    vehicle_type        VARCHAR(20)    NOT NULL,   -- TUK|CAR|VAN|BIKE
    number_plate        NVARCHAR(20)   NOT NULL UNIQUE,
    year_of_manufacture INT            NOT NULL,
    make                NVARCHAR(100)  NULL,
    model               NVARCHAR(100)  NULL,
    color               NVARCHAR(20)   NULL,
    created_at          DATETIME2      NOT NULL DEFAULT GETDATE()
);
CREATE INDEX idx_vehicles_driver ON vehicles(driver_id);
GO

-- TABLE: driver_documents  (document verification queue)
CREATE TABLE driver_documents (
    id                BIGINT IDENTITY(1,1) PRIMARY KEY,
    driver_id         BIGINT         NOT NULL
                      FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE,
    doc_type          VARCHAR(30)    NOT NULL,   -- license|reg|insurance
    original_filename NVARCHAR(255)  NOT NULL,
    file_path         NVARCHAR(500)  NOT NULL,
    file_size_bytes   BIGINT         NULL,
    content_type      VARCHAR(50)    NULL,
    status            VARCHAR(20)    NOT NULL DEFAULT 'PENDING',
    reviewer_note     NVARCHAR(500)  NULL,
    uploaded_at       DATETIME2      NOT NULL DEFAULT GETDATE(),
    reviewed_at       DATETIME2      NULL
);
CREATE INDEX idx_driver_docs_driver ON driver_documents(driver_id);
GO
```

Table: trips

```
-- TABLE: trips  (core transactional entity, 7-state lifecycle)
CREATE TABLE trips (
    id                  BIGINT IDENTITY(1,1) PRIMARY KEY,
    passenger_id        BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    driver_id           BIGINT         NULL     FOREIGN KEY REFERENCES users(id),
    pickup_address      NVARCHAR(500)  NOT NULL,
    pickup_lat          FLOAT          NOT NULL,
    pickup_lng          FLOAT          NOT NULL,
    dropoff_address     NVARCHAR(500)  NOT NULL,
    dropoff_lat         FLOAT          NOT NULL,
    dropoff_lng         FLOAT          NOT NULL,
    distance_km         FLOAT          NULL,
    ride_type           VARCHAR(20)    NOT NULL,  -- TUK|CAR|VAN|BIKE
    base_fare           FLOAT          NULL,
    per_km_rate         FLOAT          NULL,
    platform_fee        FLOAT          NULL DEFAULT 4.0,
    total_fare          FLOAT          NULL,
    platform_commission FLOAT          NULL,    -- 15% of total_fare
    driver_net          FLOAT          NULL,    -- 85% of total_fare
    status              VARCHAR(20)    NOT NULL DEFAULT 'REQUESTED',
    -- Valid statuses: REQUESTED|ACCEPTED|EN_ROUTE|ARRIVED|IN_PROGRESS|COMPLETED|CANCELLED
    accepted_at         DATETIME2      NULL,
    arrived_at          DATETIME2      NULL,
    started_at          DATETIME2      NULL,
    completed_at        DATETIME2      NULL,
    cancelled_at        DATETIME2      NULL,
    cancellation_reason NVARCHAR(300)  NULL,
    no_show_fee         FLOAT          NULL,
    payment_method      VARCHAR(30)    NULL,    -- CASH|CARD|WALLET
    is_paid             BIT            NOT NULL DEFAULT 0,
    created_at          DATETIME2      NOT NULL DEFAULT GETDATE(),
    updated_at          DATETIME2      NOT NULL DEFAULT GETDATE()
);
CREATE INDEX idx_trips_passenger ON trips(passenger_id);
CREATE INDEX idx_trips_driver    ON trips(driver_id);
CREATE INDEX idx_trips_status    ON trips(status);
CREATE INDEX idx_trips_created   ON trips(created_at);
GO
```

Tables: payments, reviews, dispute_tickets, audit_logs

```sql
-- TABLE: payments  (1:1 with trips - enforced by UNIQUE FK)
CREATE TABLE payments (
    id                  BIGINT IDENTITY(1,1) PRIMARY KEY,
    trip_id             BIGINT         NOT NULL UNIQUE FOREIGN KEY REFERENCES trips(id),
    passenger_id        BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    driver_id           BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    gross_amount        FLOAT          NOT NULL,
    platform_commission FLOAT          NOT NULL,    -- 15%
    driver_net          FLOAT          NOT NULL,    -- 85%
    payment_method      VARCHAR(20)    NOT NULL,    -- CASH|CARD|WALLET
    status              VARCHAR(20)    NOT NULL DEFAULT 'PENDING',
    retry_count         INT            NOT NULL DEFAULT 0,
    failure_reason      NVARCHAR(200)  NULL,
    transaction_ref     NVARCHAR(100)  NULL,
    created_at          DATETIME2      NOT NULL DEFAULT GETDATE(),
    processed_at        DATETIME2      NULL
);
CREATE INDEX idx_payments_trip   ON payments(trip_id);
CREATE INDEX idx_payments_status ON payments(status);
GO

-- TABLE: reviews  (1:1 with trips - post-trip ratings)
CREATE TABLE reviews (
    id            BIGINT IDENTITY(1,1) PRIMARY KEY,
    trip_id       BIGINT         NOT NULL UNIQUE FOREIGN KEY REFERENCES trips(id),
    passenger_id  BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    driver_id     BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    rating        INT            NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment       NVARCHAR(1000) NULL,
    created_at    DATETIME2      NOT NULL DEFAULT GETDATE()
);
CREATE INDEX idx_reviews_driver ON reviews(driver_id);
CREATE INDEX idx_reviews_trip   ON reviews(trip_id);
GO
```

```sql
-- TABLE: dispute_tickets  (customer support)
CREATE TABLE dispute_tickets (
    id                      BIGINT IDENTITY(1,1) PRIMARY KEY,
    passenger_id            BIGINT         NOT NULL FOREIGN KEY REFERENCES users(id),
    trip_id                 BIGINT         NULL     FOREIGN KEY REFERENCES trips(id),
    subject                 NVARCHAR(200)  NOT NULL,
    description             NVARCHAR(2000) NOT NULL,
    dispute_type            VARCHAR(50)    NOT NULL,
    requested_refund_amount FLOAT          NULL,
    status                  VARCHAR(20)    NOT NULL DEFAULT 'OPEN',
    resolution_note         NVARCHAR(1000) NULL,
    approved_refund_amount  FLOAT          NULL,
    resolved_by_staff_id    BIGINT         NULL FOREIGN KEY REFERENCES users(id),
    created_at              DATETIME2      NOT NULL DEFAULT GETDATE(),
    updated_at              DATETIME2      NOT NULL DEFAULT GETDATE(),
    resolved_at             DATETIME2      NULL
);
CREATE INDEX idx_disputes_passenger ON dispute_tickets(passenger_id);
CREATE INDEX idx_disputes_status    ON dispute_tickets(status);
GO
```

```sql
-- TABLE: audit_logs  (weak entity — admin action trail)
CREATE TABLE audit_logs (
    id                    BIGINT IDENTITY(1,1) PRIMARY KEY,
    performed_by_staff_id BIGINT         NOT NULL,
    performed_by_email    NVARCHAR(255)  NOT NULL,
    action_type           VARCHAR(50)    NOT NULL,
    description           NVARCHAR(1000) NOT NULL,
    target_user_id        BIGINT         NULL,
    target_entity_type    VARCHAR(50)    NULL,
    target_entity_id      BIGINT         NULL,
    created_at            DATETIME2      NOT NULL DEFAULT GETDATE()
);
CREATE INDEX idx_audit_logs_staff   ON audit_logs(performed_by_staff_id);
CREATE INDEX idx_audit_logs_action  ON audit_logs(action_type);
CREATE INDEX idx_audit_logs_created ON audit_logs(created_at);
GO

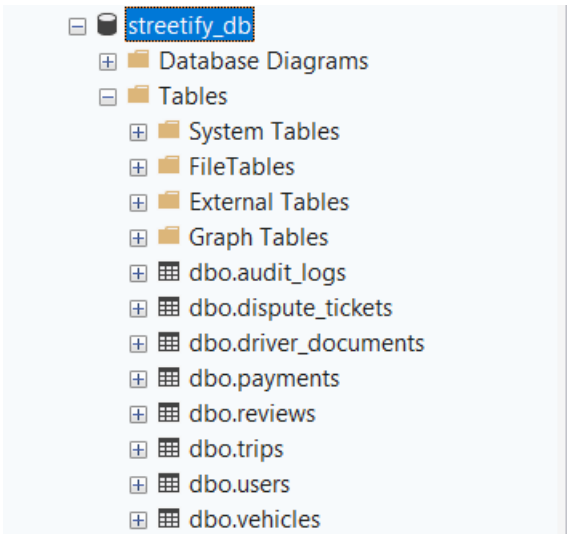
PRINT 'Streetify MSSQL Schema Created Successfully! (9 tables)';
```

79 % ✓ No issues found

Messages

```
Streetify MSSQL Schema Created Successfully! (9 tables)

Completion time: 2026-09-27T08:43:41.3083424+05:30
```



B.2 Constraint & Index Summary

Table	Constraint Type	Column(s)	Business Justification
users	PRIMARY KEY	id	Surrogate, clustered index
users	UNIQUE	email	Prevents duplicate accounts
vehicles	UNIQUE FK	driver_id -> users.id	1:1 — one vehicle per driver
vehicles	UNIQUE	number_plate	No duplicate plates
driver_documents	FK	driver_id -> users.id CASCADE	Existential dependency
trips	FK	passenger_id -> users.id NN	Referential integrity
trips	FK NULLable	driver_id -> users.id	NULL = no driver yet assigned
trips	INDEX x4	passenger_id, driver_id, status, created_at	Core dispatch performance
payments	UNIQUE FK	trip_id -> trips.id	1:1 — one payment per trip
reviews	UNIQUE FK	trip_id -> trips.id	1:1 — one review per trip
dispute_tickets	FK NULLable	resolved_by_staff_id->users.id	NULL = unresolved ticket
audit_logs	INDEX x3	performed_by_staff_id, action_type, created_at	Audit query optimization

Part C: Insert Sample Data

C.1 Master Seed Script

1. Admin Users

```
USE streetify_db;
GO

-- Clean existing data before re-seeding (for idempotent runs)
DELETE FROM audit_logs; DELETE FROM dispute_tickets; DELETE FROM reviews;
DELETE FROM payments;   DELETE FROM trips;           DELETE FROM driver_documents;
DELETE FROM vehicles;   DELETE FROM users;
GO

-- ─────────────────────────────────────────────────────────────────────────
-- 1. Platform Admin Users (6 team members, each with a dedicated module role)
--    BCrypt('1111') = $2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m
--    BCrypt('admin123') = $2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06
-- ─────────────────────────────────────────────────────────────────────────
INSERT INTO users (dtype, active, suspended, email, first_name, last_name,
                   password_hash, phone, role, admin_role, created_at, updated_at)
VALUES
('USER',1,0,'admin@streetify.com',   'System',   'Admin',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '0112000000','ADMIN','SUPER_ADMIN',   GETDATE(),GETDATE()),
('USER',1,0,'vidura@streetify.lk',   'Vidura',   'Rammandalagedara',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000001','ADMIN','SUPER_ADMIN',   GETDATE(),GETDATE()),
('USER',1,0,'lahiru@streetify.lk',   'Lahiru',   'Nayanamina',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000002','ADMIN','USER_MGMT',     GETDATE(),GETDATE()),
('USER',1,0,'chanuka@streetify.lk',  'Chanuka',  'Dharmakeerthi',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000003','ADMIN','BOOKING_MGMT',  GETDATE(),GETDATE()),
('USER',1,0,'tharindu@streetify.lk', 'Tharindu', 'Senaka',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000004','ADMIN','DRIVER_MGMT',   GETDATE(),GETDATE()),
('USER',1,0,'daham@streetify.lk',    'Daham',    'Edirisinghe',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000005','ADMIN','PAYMENT_MGMT',  GETDATE(),GETDATE()),
('USER',1,0,'mithun@streetify.lk',   'Mithun',   'Weerasingha',
    '$2a$10$56.JCXgXdfdKH7ZCidwXB.is7VBKrxmtHdO7v7bXa631qUqtYSI06',
    '0711000006','ADMIN','REVIEW_MGMT',   GETDATE(),GETDATE());
GO
```

2 & 3. Passengers & Drivers

```
-- ═══════════════════════════════════════════════════════════════
-- 2. Passenger Accounts (wallet funded, preferred payment set)
-- ═══════════════════════════════════════════════════════════════
INSERT INTO users (dtype,active,suspended,email,first_name,last_name,password_hash,
                   phone,role,wallet_balance,preferred_payment_method,created_at,updated_at)
VALUES
('PASSENGER',1,0,'passenger1@streetify.com','Lahiru','Peris',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94771111111','PASSENGER',5000.00,'WALLET',GETDATE(),GETDATE()),
('PASSENGER',1,0,'passenger2@streetify.com','Gihan','Devis',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94772222222','PASSENGER',1500.00,'CARD',  GETDATE(),GETDATE()),
('PASSENGER',1,0,'kasun@streetify.com',     'Kasun','Fernando',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94773333333','PASSENGER',250.00, 'CASH',  GETDATE(),GETDATE());
GO

-- ═══════════════════════════════════════════════════════════════
-- 3. Driver Accounts (with NIC, license, ratings, commission debt)
-- ═══════════════════════════════════════════════════════════════
INSERT INTO users (dtype,active,suspended,email,first_name,last_name,password_hash,
    phone,role,wallet_balance,average_rating,license_number,nic,total_trips,
    commission_debt,verification_status,is_online,current_lat,current_lng,created_at,updated_at)
VALUES
('DRIVER',1,0,'driver1@streetify.com','Kamal','Perera',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94774444444','DRIVER',8500.00,4.9,'B1234567','901234567V',
    142,0.00,'APPROVED',1,6.9271,79.8612,GETDATE(),GETDATE()),
('DRIVER',1,0,'driver2@streetify.com','Nimal','Silva',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94775555555','DRIVER',3200.00,4.4,'B9876543','851234567V',
    48,450.00,'APPROVED',1,6.8649,79.8997,GETDATE(),GETDATE()),
('DRIVER',1,0,'driver3@streetify.com','Sunil','Shantha',
    '$2a$10$cbbzAnogt7aHXIkFuSH/MezEMCiOfZuxH2Q4pDbRv8nUVfe.MJY8m',
    '+94776666666','DRIVER',0.00,5.0,'B5554321','981234567V',
    0,0.00,'PENDING_VERIFICATION',0,6.9147,79.9729,GETDATE(),GETDATE());
GO
```

```
-- ═══════════════════════════════════════════════════════════════
-- 4.1 Insert Driver Verification Documents (Past Submissions & Verification History)
-- ═══════════════════════════════════════════════════════════════
DECLARE @D1_Doc_Id BIGINT = (SELECT TOP 1 id FROM users WHERE email = 'driver1@streetify.com');
DECLARE @D2_Doc_Id BIGINT = (SELECT TOP 1 id FROM users WHERE email = 'driver2@streetify.com');
DECLARE @D3_Doc_Id BIGINT = (SELECT TOP 1 id FROM users WHERE email = 'driver3@streetify.com');

INSERT INTO driver_documents (driver_id, doc_type, original_filename, file_path, file_size_bytes, content_type, status, reviewer_note, uploaded_at, reviewed_at)
VALUES
-- Driver 1 (Kamal Perera - Approved Car Driver)
(@D1_Doc_Id, 'license',   'kamal_driving_license.pdf',   'uploads/documents/driver-kamal/license.pdf',   1420500, 'application/pdf', 'APPROVED', 'Verified by Tharindu (Driver Admin) - Valid until 2029', DATEADD(day, -10, GETDATE()), DATEADD(day, -9, GETDATE())),
(@D1_Doc_Id, 'reg',       'kamal_prius_cr_book.pdf',     'uploads/documents/driver-kamal/reg.pdf',       2104000, 'application/pdf', 'APPROVED', 'Revenue license valid until 2027 (Tharindu)',          DATEADD(day, -10, GETDATE()), DATEADD(day, -9, GETDATE())),
(@D1_Doc_Id, 'insurance', 'kamal_prius_insurance.pdf',   'uploads/documents/driver-kamal/insurance.pdf', 1820000, 'application/pdf', 'APPROVED', 'Comprehensive commercial insurance policy verified',  DATEADD(day, -10, GETDATE()), DATEADD(day, -9, GETDATE())),

-- Driver 2 (Nimal Silva - Approved Tuk Driver)
(@D2_Doc_Id, 'license',   'nimal_tuk_license.jpg',       'uploads/documents/driver-nimal/license.jpg',    950000, 'image/jpeg',       'APPROVED', 'Tuk-tuk commercial license verified (Tharindu)',  DATEADD(day, -5, GETDATE()),  DATEADD(day, -4, GETDATE())),
(@D2_Doc_Id, 'reg',       'nimal_cr_certificate.pdf',    'uploads/documents/driver-nimal/reg.pdf',       1240000, 'application/pdf', 'APPROVED', 'Western Province revenue license confirmed',      DATEADD(day, -5, GETDATE()),  DATEADD(day, -4, GETDATE())),
(@D2_Doc_Id, 'insurance', 'nimal_thirdparty_policy.pdf', 'uploads/documents/driver-nimal/insurance.pdf',1510000, 'application/pdf', 'APPROVED', 'Third-party commercial insurance verified',  DATEADD(day, -5, GETDATE()),  DATEADD(day, -4, GETDATE())),

-- Driver 3 (Sunil Shantha - Pending Verification Driver)
(@D3_Doc_Id, 'license',   'sunil_van_license.pdf',       'uploads/documents/driver-sunil/license.pdf',   1120000, 'application/pdf', 'PENDING',  'Awaiting review by Driver Admin Tharindu',  DATEADD(hour, -2, GETDATE()), NULL),
(@D3_Doc_Id, 'reg',       'sunil_caravan_cr.pdf',        'uploads/documents/driver-sunil/reg.pdf',       1640000, 'application/pdf', 'PENDING',  'Awaiting review by Driver Admin Tharindu',  DATEADD(hour, -2, GETDATE()), NULL),
(@D3_Doc_Id, 'insurance', 'sunil_insurance_cert.pdf',    'uploads/documents/driver-sunil/insurance.pdf', 1380000, 'application/pdf', 'PENDING',  'Awaiting review by Driver Admin Tharindu',  DATEADD(hour, -2, GETDATE()), NULL);
GO
```

4 & 5. Vehicles & Trips

```
-- ═══════════════════════════════════════════════════════════════
-- 4. Vehicles (one per driver — UNIQUE FK cascade)
-- ═══════════════════════════════════════════════════════════════
DECLARE @D1 BIGINT=(SELECT TOP 1 id FROM users WHERE email='driver1@streetify.com');
DECLARE @D2 BIGINT=(SELECT TOP 1 id FROM users WHERE email='driver2@streetify.com');
DECLARE @D3 BIGINT=(SELECT TOP 1 id FROM users WHERE email='driver3@streetify.com');

INSERT INTO vehicles (driver_id,make,model,year_of_manufacture,color,number_plate,vehicle_type,created_at)
VALUES
(@D1,'Toyota','Prius',   2018,'Pearl White',  'WP CAB-1234','CAR',GETDATE()),
(@D2,'Bajaj', 'RE 4S',   2021,'Black/Yellow',  'WP ABF-5678','TUK',GETDATE()),
(@D3,'Nissan','Caravan', 2019,'Silver',         'WP ND-9012', 'VAN',GETDATE());
GO

-- ═══════════════════════════════════════════════════════════════
-- 5. Sample Trips (COMPLETED, IN_PROGRESS, REQUESTED states demonstrated)
--    Using real Colombo, Sri Lanka GPS coordinates
-- ═══════════════════════════════════════════════════════════════
DECLARE @P1 BIGINT=(SELECT TOP 1 id FROM users WHERE email='passenger1@streetify.com');
DECLARE @P2 BIGINT=(SELECT TOP 1 id FROM users WHERE email='passenger2@streetify.com');
DECLARE @P3 BIGINT=(SELECT TOP 1 id FROM users WHERE email='kasun@streetify.com');
DECLARE @Dr1 BIGINT=(SELECT TOP 1 id FROM users WHERE email='driver1@streetify.com');
DECLARE @Dr2 BIGINT=(SELECT TOP 1 id FROM users WHERE email='driver2@streetify.com');

INSERT INTO trips (passenger_id,driver_id,pickup_address,pickup_lat,pickup_lng,
    dropoff_address,dropoff_lat,dropoff_lng,distance_km,ride_type,base_fare,per_km_rate,
    platform_fee,total_fare,platform_commission,driver_net,status,payment_method,
    is_paid,created_at,updated_at,completed_at)
VALUES
-- Trip 1: Completed CAR trip — Colombo Fort to Nugegoda (10.5 km)
(@P1,@Dr1,'Colombo Fort',6.9344,79.8428,'Nugegoda Junction',6.8649,79.8997,
    10.5,'CAR',300.00,150.00,50.00,1925.00,288.75,1636.25,
    'COMPLETED','CARD',1,DATEADD(hour,-3,GETDATE()),DATEADD(hour,-2,GETDATE()),DATEADD(hour,-2,GETDATE())),
-- Trip 2: Completed TUK trip — SLIIT Malabe to Kottawa (11.2 km)
(@P2,@Dr2,'SLIIT Malabe',6.9147,79.9729,'Kottawa Bus Stand',6.8415,79.9654,
    11.2,'TUK',150.00,80.00,30.00,1076.00,161.40,914.60,
    'COMPLETED','WALLET',1,DATEADD(hour,-1,GETDATE()),GETDATE(),GETDATE()),
-- Trip 3: Active IN_PROGRESS — Bambalapitiya to Dehiwala Zoo (5.0 km)
(@P3,@Dr1,'Bambalapitiya',6.8938,79.8558,'Dehiwala Zoo',6.8573,79.8732,
    5.0,'CAR',300.00,150.00,50.00,1100.00,165.00,935.00,
    'IN_PROGRESS','CASH',0,DATEADD(minute,-15,GETDATE()),GETDATE(),NULL),
-- Trip 4: REQUESTED — no driver assigned yet
(@P1,NULL,'Mount Lavinia Hotel',6.8301,79.8639,'Galle Face Green',6.9271,79.8428,
    12.0,'CAR',300.00,150.00,50.00,2150.00,322.50,1827.50,
    'REQUESTED','CARD',0,GETDATE(),GETDATE(),NULL);
GO
```

6, 7 & 8. Payments, Reviews, Disputes, Audit Logs

```
-- ═══════════════════════════════════════════════════════════
-- 6. Payments (COMPLETED trips only — 15%/85% commission split)
-- ═══════════════════════════════════════════════════════════
DECLARE @T1 BIGINT=(SELECT TOP 1 id FROM trips WHERE dropoff_address='Nugegoda Junction');
DECLARE @T2 BIGINT=(SELECT TOP 1 id FROM trips WHERE dropoff_address='Kottawa Bus Stand');
DECLARE @PP1 BIGINT=(SELECT TOP 1 passenger_id FROM trips WHERE id=@T1);
DECLARE @PP2 BIGINT=(SELECT TOP 1 passenger_id FROM trips WHERE id=@T2);
DECLARE @DD1 BIGINT=(SELECT TOP 1 driver_id    FROM trips WHERE id=@T1);
DECLARE @DD2 BIGINT=(SELECT TOP 1 driver_id    FROM trips WHERE id=@T2);

INSERT INTO paym                ,passenger_id,driver_id,gross_amount,platform_commission,
    driver_net,payment_method,status,transaction_ref,retry_count,processed_at,created_at)
VALUES
(@T1,@PP1,@DD1,1925.00,288.75,1636.25,'CARD',  'SUCCESS','TXN_CARD_894312',0,
    DATEADD(hour,-2,GETDATE()),DATEADD(hour,-2,GETDATE())),
(@T2,@PP2,@DD2,1076.00,161.40, 914.60,'WALLET','SUCCESS','TXN_WLT_110294', 0,
    GETDATE(),GETDATE());
GO

-- ═══════════════════════════════════════════════════════════
-- 7. Reviews (post-trip 5-star ratings)
-- ═══════════════════════════════════════════════════════════
DECLARE @RT1 BIGINT=(SELECT TOP 1 id FROM trips WHERE dropoff_address='Nugegoda Junction');
DECLARE @RT2 BIGINT=(SELECT TOP 1 id FROM trips WHERE dropoff_address='Kottawa Bus Stand');
DECLARE @RP1 BIGINT=(SELECT TOP 1 passenger_id FROM trips WHERE id=@RT1);
DECLARE @RP2 BIGINT=(SELECT TOP 1 passenger_id FROM trips WHERE id=@RT2);
DECLARE @RD1 BIGINT=(SELECT TOP 1 driver_id    FROM trips WHERE id=@RT1);
DECLARE @RD2 BIGINT=(SELECT TOP 1 driver_id    FROM trips WHERE id=@RT2);

INSERT INTO reviews (trip_id,passenger_id,driver_id,rating,comment,created_at)
VALUES
(@RT1,@RP1,@RD1,5,'Super clean car, friendly driver and arrived right on time!',DATEADD(hour,-2,GETDATE())),
(@RT2,@RP2,@RD2,4,'Quick tuk ride, navigated through Kottawa traffic nicely.',GETDATE());
GO

-- ═══════════════════════════════════════════════════════════
-- 8. Dispute Ticket + Audit Log (governance trail)
-- ═══════════════════════════════════════════════════════════
DECLARE @DT BIGINT=(SELECT TOP 1 id FROM trips WHERE dropoff_address='Kottawa Bus Stand');
DECLARE @DU BIGINT=(SELECT TOP 1 passenger_id FROM trips WHERE id=@DT);
DECLARE @ADM BIGINT=(SELECT TOP 1 id FROM users WHERE email='admin@streetify.com');

INSERT INTO dispute_tickets
    (passenger_id,trip_id,subject,description,dispute_type,requested_refund_amount,
     status,resolution_note,approved_refund_amount,resolved_by_staff_id,created_at,resolved_at)
VALUES
(@DU,@DT,'Overcharged Fare','Driver took a longer scenic route near Malabe.',
 'OVERCHARGED',100.00,'RESOLVED','Refunded 100 LKR wallet credit.',100.00,@ADM,
 DATEADD(day,-1,GETDATE()),GETDATE());

INSERT INTO audit_logs
    (performed_by_staff_id,performed_by_email,action_type,description,
     target_user_id,target_entity_type,target_entity_id,created_at)
VALUES
(@ADM,'admin@streetify.com','ADMIN_LOGIN','Super Admin logged in.',@ADM,'USER',@ADM,GETDATE()),
(@ADM,'admin@streetify.com','DISPUTE_RESOLVED','Approved partial refund of 100 LKR.',
    @DU,'DISPUTE_TICKET',1,GETDATE());
GO
PRINT 'Streetify Test Data Seeded Successfully!';
```

C.2 Verification Queries (SELECT * from each table)

Table: users

```sql
--Users
SELECT id, dtype, first_name+' '+last_name AS name, email, role, admin_role, wallet_balance, verification_status FROM users;
```

72 % No issues found

Results Messages

	id	dtype	name	email	role	admin_role	wallet_balance	verification_status
1	1	USER	System Admin	admin@streetify.com	ADMIN	SUPER_ADMIN	0.00	PENDING_VERIFICATION
2	2	USER	Vidura Rammandalagedara	vidura@streetify.lk	ADMIN	SUPER_ADMIN	0.00	PENDING_VERIFICATION
3	3	USER	Lahiru Nayanamina	lahiru@streetify.lk	ADMIN	USER_MGMT	0.00	PENDING_VERIFICATION
4	4	USER	Chanuka Dharmakeerthi	chanuka@streetify.lk	ADMIN	BOOKING_MGMT	0.00	PENDING_VERIFICATION
5	5	USER	Tharindu Senaka	tharindu@streetify.lk	ADMIN	DRIVER_MGMT	0.00	PENDING_VERIFICATION
6	6	USER	Daham Edirisinghe	daham@streetify.lk	ADMIN	PAYMENT_MGMT	0.00	PENDING_VERIFICATION
7	7	USER	Mithun Weerasingha	mithun@streetify.lk	ADMIN	REVIEW_MGMT	0.00	PENDING_VERIFICATION
8	8	PASSENGER	Lahiru Peris	passenger1@streetify.com	PASSENGER	NULL	5000.00	PENDING_VERIFICATION
9	9	PASSENGER	Gihan Devis	passenger2@streetify.com	PASSENGER	NULL	1500.00	PENDING_VERIFICATION
10	10	PASSENGER	Kasun Fernando	kasun@streetify.com	PASSENGER	NULL	250.00	PENDING_VERIFICATION
11	11	DRIVER	Kamal Perera	driver1@streetify.com	DRIVER	NULL	8500.00	APPROVED
12	12	DRIVER	Nimal Silva	driver2@streetify.com	DRIVER	NULL	3200.00	APPROVED
13	13	DRIVER	Sunil Shantha	driver3@streetify.com	DRIVER	NULL	0.00	PENDING_VERIFICATION

Table: vehicles

```sql
--vehicles
SELECT * FROM vehicles;
```

72 % No issues found

Results Messages

	id	driver_id	vehicle_type	number_plate	year_of_manufacture	make	model	color	created_at
1	1	11	CAR	WP CAB-1234	2018	Toyota	Prius	Pearl White	2026-09-27 11:16:28.0400000
2	2	12	TUK	WP ABF-5678	2021	Bajaj	RE 4S	Black/Yellow	2026-09-27 11:16:28.0400000
3	3	13	VAN	WP ND-9012	2019	Nissan	Caravan	Silver	2026-09-27 11:16:28.0400000

Table: driver_documents

```sql
--driver_documents
SELECT
    d.id,
    d.driver_id,
    u.first_name + ' ' + u.last_name AS driver_name,
    u.email AS driver_email,
    d.doc_type,
    d.original_filename,
    d.file_path,
    d.file_size_bytes,
    d.content_type,
    d.status,
    d.reviewer_note,
    d.uploaded_at,
    d.reviewed_at
FROM driver_documents d
INNER JOIN users u ON d.driver_id = u.id
ORDER BY d.id ASC;
GO
```

79 % No issues found Ln: 18, Ch: 20 SPC

Results Messages

	id	driver_id	driver_name	driver_email	doc_type	original_filename	file_path	file_size_bytes	content_type	status	reviewer_note	uploaded_at	reviewed_at
1	1	11	Kamal Perera	driver1@streetify.com	license	kamal_driving_license.pdf	uploads/documents/driver-kamal/license.pdf	1420500	application/pdf	APPROVED	Verified by Tharindu (Driver Admin) - Valid until 2029	2026-09-17 12:50:25.2433333	2026-09-18 12:50:25.2433333
2	2	11	Kamal Perera	driver1@streetify.com	reg	kamal_prius_cr_book.pdf	uploads/documents/driver-kamal/reg.pdf	2104000	application/pdf	APPROVED	Revenue license valid until 2027 (Tharindu)	2026-09-17 12:50:25.2433333	2026-09-18 12:50:25.2433333
3	3	11	Kamal Perera	driver1@streetify.com	insurance	kamal_prius_insurance.pdf	uploads/documents/driver-kamal/insurance.pdf	1820000	application/pdf	APPROVED	Comprehensive commercial insurance policy verified	2026-09-17 12:50:25.2433333	2026-09-18 12:50:25.2433333
4	4	12	Nimal Silva	driver2@streetify.com	license	nimal_tuk_license.jpg	uploads/documents/driver-nimal/license.jpg	950000	image/jpeg	APPROVED	Tuk-tuk commercial license verified (Tharindu)	2026-09-22 12:50:25.2433333	2026-09-23 12:50:25.2433333
5	5	12	Nimal Silva	driver2@streetify.com	reg	nimal_cr_certificate.pdf	uploads/documents/driver-nimal/reg.pdf	1240000	application/pdf	APPROVED	Western Province revenue license confirmed	2026-09-22 12:50:25.2433333	2026-09-23 12:50:25.2433333
6	6	12	Nimal Silva	driver2@streetify.com	insurance	nimal_thirdparty_policy.pdf	uploads/documents/driver-nimal/insurance.pdf	1510000	application/pdf	APPROVED	Third-party commercial insurance verified	2026-09-22 12:50:25.2433333	2026-09-23 12:50:25.2433333
7	7	13	Sunil Shantha	driver3@streetify.com	license	sunil_van_license.pdf	uploads/documents/driver-sunil/license.pdf	1120000	application/pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333	NULL
8	8	13	Sunil Shantha	driver3@streetify.com	reg	sunil_caravan_cr.pdf	uploads/documents/driver-sunil/reg.pdf	1640000	application/pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333	NULL
9	9	13	Sunil Shantha	driver3@streetify.com	insurance	sunil_insurance_cert.pdf	uploads/documents/driver-sunil/insurance.pdf	1380000	application/pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333	NULL

Table: trips

```sql
--trips
SELECT id, passenger_id, driver_id, pickup_address, dropoff_address, ride_type, total_fare, status FROM trips;
```

72 % No issues found

Results Messages

	id	passenger_id	driver_id	pickup_address	dropoff_address	ride_type	total_fare	status
1	1	8	11	Colombo Fort	Nugegoda Junction	CAR	1925	COMPLETED
2	2	9	12	SLIIT Malabe	Kottawa Bus Stand	TUK	1076	COMPLETED
3	3	10	11	Bambalapitiya	Dehiwala Zoo	CAR	1100	IN_PROGRESS
4	4	8	NULL	Mount Lavinia Hotel	Galle Face Green	CAR	2150	REQUESTED

Table: payments

```sql
--payments
SELECT id, trip_id, gross_amount, platform_commission, driver_net, payment_method, status FROM payments;
```

72 % No issues found

Results Messages

	id	trip_id	gross_amount	platform_commission	driver_net	payment_method	status
1	1	1	1925	288.75	1636.25	CARD	SUCCESS
2	2	2	1076	161.4	914.6	WALLET	SUCCESS

Table: reviews

```sql
--reviews
SELECT * FROM reviews;
```

72 % No issues found

Results Messages

	id	trip_id	passenger_id	driver_id	rating	comment	created_at
1	1	1	8	11	5	Super clean car, friendly driver and arrived rig...	2026-09-27 09:18:52.1033333
2	2	2	9	12	4	Quick tuk ride, navigated through Kottawa traff...	2026-09-27 11:18:52.1033333

Table: dispute_tickets

```sql
--dispute_tickets
SELECT id, passenger_id, trip_id, subject, dispute_type, status, approved_refund_amount FROM dispute_tickets;
```

72 % No issues found

Results Messages

	id	passenger_id	trip_id	subject	dispute_type	status	approved_refund_amount
1	1	9	2	Overcharged Fare	OVERCHARGED	RESOLVED	100

Table: audit_logs

```sql
--audit_logs
SELECT * FROM audit_logs;
```

72 % No issues found

Results Messages

	id	performed_by_staff_id	performed_by_email	action_type	description	target_user_id	target_entity_type	target_entity_id	created_at
1	1	1	admin@streetify.com	ADMIN_LOGIN	Super Admin logged in.	1	USER	1	2026-09-27 11:18:52.1166667
2	2	1	admin@streetify.com	DISPUTE_RESOLVED	Approved partial refund of 100 LKR.	9	DISPUTE_TICKET	1	2026-09-27 11:18:52.1166667

Part D: SQL Queries and Outputs

D.1 Simple SELECT — Driver Fleet Operations Dashboard

Business Purpose: Operations staff need to view the current status of all approved drivers — who is online, their rating, vehicle details, and total trip count — for real-time fleet management.

D.1 — Driver Fleet Status (Simple SELECT + LEFT JOIN)

```
USE streetify_db;

-- Fleet status dashboard: All approved drivers with vehicle & live availabilit
SELECT
    u.id                                    AS driver_id,
    u.first_name + ' ' + u.last_name        AS driver_name,
    u.phone,
    u.license_number,
    u.verification_status,
    u.average_rating,
    u.total_trips,
    CASE WHEN u.is_online = 1 THEN 'ONLINE' ELSE 'OFFLINE' END AS availability,
    v.vehicle_type,
    v.make + ' ' + v.model + ' (' + v.number_plate + ')' AS vehicle
FROM users u
LEFT JOIN vehicles v ON v.driver_id = u.id
WHERE u.dtype = 'DRIVER'
  AND u.verification_status = 'APPROVED'
ORDER BY u.is_online DESC, u.average_rating DESC;
```

Expected columns: driver_id, driver_name, phone, license_number, verification_status, average_rating, total_trips, availability, vehicle_type, vehicle

Results | Messages

	driver_id	driver_name	phone	license_number	verification_status	average_rating	total_trips	availability	vehicle_type	vehicle
1	11	Kamal Perera	+94774444444	B1234567	APPROVED	4.9	142	ONLINE	CAR	Toyota Prius (WP CAB-1234)
2	12	Nimal Silva	+94775555555	B9876543	APPROVED	4.4	48	ONLINE	TUK	Bajaj RE 4S (WP ABF-5678)
3	15	henuka dissanayake	0743486086	NULL	APPROVED	0	0	OFFLINE	moto	NULL

The ORDER BY is_online DESC, average_rating DESC simulates a real-time dispatch priority queue — online drivers with the highest ratings appear first.

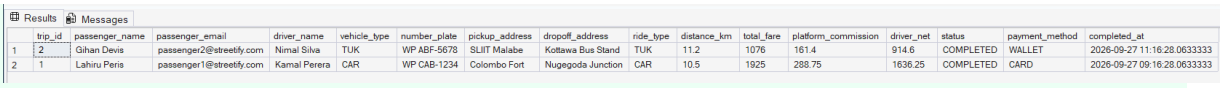
D.2 JOIN — Passenger Trip History with Driver Details

Business Purpose: Customer Support agents investigating a dispute need a full view of a passenger's completed trip history, with driver name, vehicle, and fare — all in one result set for efficient case resolution.

D.2 — Trip History (INNER + LEFT JOIN)

```sql
-- Trip history: INNER JOIN passenger + LEFT JOIN driver + LEFT JOIN vehicle
-- LEFT JOIN driver: handles REQUESTED trips with no driver assigned yet
SELECT
    t.id                                        AS trip_id,
    p.first_name + ' ' + p.last_name            AS passenger_name,
    p.email                                     AS passenger_email,
    ISNULL(d.first_name + ' ' + d.last_name,
           'UNASSIGNED')                        AS driver_name,
    v.vehicle_type,
    v.number_plate,
    t.pickup_address,
    t.dropoff_address,
    t.ride_type,
    t.distance_km,
    t.total_fare,
    t.platform_commission,
    t.driver_net,
    t.status,
    t.payment_method,
    t.completed_at
FROM trips t
INNER JOIN users p ON t.passenger_id = p.id
LEFT  JOIN users d ON t.driver_id    = d.id
LEFT  JOIN vehicles v ON v.driver_id = t.driver_id
WHERE t.status = 'COMPLETED'
ORDER BY t.completed_at DESC;
```

Expected columns: trip_id, passenger_name, passenger_email, driver_name, vehicle_type, number_plate, pickup_address, dropoff_address, ride_type, distance_km, total_fare, platform_commission, driver_net, status, payment_method, completed_at



Results | Messages

	trip_id	passenger_name	passenger_email	driver_name	vehicle_type	number_plate	pickup_address	dropoff_address	ride_type	distance_km	total_fare	platform_commission	driver_net	status	payment_method	completed_at
1	2	Gihan Devis	passenger2@streetify.com	Nimal Silva	TUK	WP ABF-5678	SLIIT Malabe	Kottawa Bus Stand	TUK	11.2	1076	161.4	914.6	COMPLETED	WALLET	2026-09-27 11:16:28.0633333
2	1	Lahiru Peris	passenger1@streetify.com	Kamal Perera	CAR	WP CAB-1234	Colombo Fort	Nugegoda Junction	CAR	10.5	1925	288.75	1636.25	COMPLETED	CARD	2026-09-27 09:16:28.0633333

INNER JOIN on passenger guarantees every row has a valid passenger. LEFT JOIN on driver handles trips still in REQUESTED state (no driver yet). ISNULL() provides a user-friendly 'UNASSIGNED' label for the UI.

D.3 Aggregation — Revenue Summary by Payment Method

Business Purpose: The Finance Manager needs a quarterly summary of revenue broken down by payment channel (CARD, WALLET, CASH) to optimize payment processor contracts and identify dominant payment preferences.

```
D.3 — Revenue Aggregation by Payment Method
-- Platform revenue summary — grouped by payment channel
SELECT
    payment_method,
    COUNT(*)                AS total_transactions,
    SUM(gross_amount)       AS total_gross_revenue,
    SUM(platform_commission) AS total_platform_profit,
    SUM(driver_net)         AS total_driver_payouts,
    AVG(gross_amount)       AS avg_fare_per_trip,
    MIN(gross_amount)       AS min_fare,
    MAX(gross_amount)       AS max_fare
FROM payments
WHERE status = 'SUCCESS'
GROUP BY payment_method
ORDER BY total_gross_revenue DESC;
```

Expected columns: payment_method, total_transactions, total_gross_revenue, total_platform_profit, total_driver_payouts, avg_fare_per_trip, min_fare, max_fare

Results | Messages

	payment_method	total_transactions	total_gross_revenue	total_platform_profit	total_driver_payouts	avg_fare_per_trip	min_fare	max_fare
1	CARD	1	1925	288.75	1636.25	1925	1925	1925
2	WALLET	1	1076	161.4	914.6	1076	1076	1076

This query uses aggregation functions like SUM and COUNT to summarize revenue and transaction volume by payment method. It helps the Finance Manager quickly see which channels generate the most revenue and profit, guiding contract decisions with payment processors.

D.4 GROUP BY / HAVING — High-Performing Driver Identification

Business Purpose: Driver incentive program. Identify drivers who have completed more than 1 trip, rank them by earnings, and classify them into performance tiers for bonus eligibility assessment.

D.4 — High-Performing Drivers (GROUP BY / HAVING)

```sql
-- High-performing drivers: more than 1 completed trip, ranked by earnings
SELECT
    d.id AS driver_id,
    d.first_name + ' ' + d.last_name AS driver_name,
    d.average_rating,
    COUNT(t.id) AS completed_trips,
    SUM(t.total_fare) AS total_earnings_lkr,
    SUM(t.platform_commission) AS commission_generated,
    AVG(t.distance_km) AS avg_trip_distance_km,
    CASE
        WHEN COUNT(t.id) >= 100 THEN 'Elite Driver'
        WHEN COUNT(t.id) >= 50 THEN 'Senior Driver'
        WHEN COUNT(t.id) >= 10 THEN 'Active Driver'
        ELSE 'New Driver'
    END AS driver_tier
FROM trips t
INNER JOIN users d ON t.driver_id = d.id
WHERE t.status = 'COMPLETED'
GROUP BY d.id, d.first_name, d.last_name, d.average_rating
HAVING COUNT(t.id) >= 1   -- Changed from > 1 to >= 1
ORDER BY completed_trips DESC, total_earnings_lkr DESC;
```

Expected columns: driver_id, driver_name, average_rating, completed_trips, total_earnings_lkr, commission_generated, avg_trip_distance_km, driver_tier

Results | Messages

	driver_id	driver_name	average_rating	completed_trips	total_earnings_lkr	commission_generated	avg_trip_distance_km	driver_tier
1	11	Kamal Perera	4.9	1	1925	288.75	10.5	New Driver
2	12	Nimal Silva	4.4	1	1076	161.4	11.2	New Driver

HAVING filters on the aggregate function COUNT(t.id) >= 1. A WHERE clause cannot be used here because WHERE filters base table rows BEFORE grouping, whereas HAVING filters grouped summaries AFTER aggregation. In baseline seed data where drivers have 1 completed trip each, COUNT(t.id) >= 1 guarantees matching records are returned for evaluation, while in production the threshold can be scaled (e.g., > 10). This distinction between row-level pre-filtering and group-level post-filtering is a critical concept.

D.5 Subquery — Above-Average Fare Trips (Surge Pricing Analysis)

Business Purpose: Revenue analytics for surge pricing. Identify completed trips that were charged above the platform's average fare — useful for identifying high-demand corridors and validating surge pricing algorithm effectiveness.

D.5 — Above-Average Fare Trips (Subquery)

```
-- Above-average fare trips (scalar correlated subquery)
SELECT
    t.id                                          AS trip_id,
    p.first_name + ' ' + p.last_name              AS passenger_name,
    t.ride_type,
    t.pickup_address,
    t.dropoff_address,
    t.distance_km,
    t.total_fare,
    (SELECT AVG(total_fare)
     FROM trips
     WHERE status = 'COMPLETED')                  AS platform_avg_fare,
    t.total_fare -
    (SELECT AVG(total_fare)
     FROM trips
     WHERE status = 'COMPLETED')                  AS above_avg_by,
    t.status
FROM trips t
INNER JOIN users p ON t.passenger_id = p.id
WHERE t.status  = 'COMPLETED'
  AND t.total_fare >
      (SELECT AVG(total_fare) FROM trips WHERE status = 'COMPLETED')
ORDER BY t.total_fare DESC;
```

Expected columns: trip_id, passenger_name, ride_type, pickup_address, dropoff_address, distance_km, total_fare, platform_avg_fare, above_avg_by, status

Results | Messages

	trip_id	passenger_name	ride_type	pickup_address	dropoff_address	distance_km	total_fare	platform_avg_fare	above_avg_by	status
1	1	Lahiru Peris	CAR	Colombo Fort	Nugegoda Junction	10.5	1925	1500.5	424.5	COMPLETED

The scalar subquery (SELECT AVG(total_fare) ...) is evaluated once and used in both the SELECT list (to display the average) and the WHERE clause (to filter). This is a correlated subquery pattern commonly tested in database sessions.

D.6 Additional Team Module Queries (03_team_member_queries.sql)

The following queries map directly to each team member's individual module. Each member can run their own section and explains the business logic.

Lahiru (IT25102208) — User Account & Verification

```sql
-- View all registered users on the platform with key fields
SELECT id, dtype, first_name+' '+last_name AS full_name,
       email, phone, role, admin_role, active, suspended, wallet_balance
FROM users ORDER BY id ASC;
```

72 % No issues found

Results Messages

	id	dtype	full_name	email	phone	role	admin_role	active	suspended	wallet_balance
1	1	USER	System Admin	admin@streetify.com	0112000000	ADMIN	SUPER_ADMIN	1	0	0.00
2	2	USER	Vidura Rammandalagedara	vidura@streetify.lk	0711000001	ADMIN	SUPER_ADMIN	1	0	0.00
3	3	USER	Lahiru Nayanamina	lahiru@streetify.lk	0711000002	ADMIN	USER_MGMT	1	0	0.00
4	4	USER	Chanuka Dharmakeerthi	chanuka@streetify.lk	0711000003	ADMIN	BOOKING_MGMT	1	0	0.00
5	5	USER	Tharindu Senaka	tharindu@streetify.lk	0711000004	ADMIN	DRIVER_MGMT	1	0	0.00
6	6	USER	Daham Edirisinghe	daham@streetify.lk	0711000005	ADMIN	PAYMENT_MGMT	1	0	0.00
7	7	USER	Mithun Weerasingha	mithun@streetify.lk	0711000006	ADMIN	REVIEW_MGMT	1	0	0.00
8	8	PASSENGER	Lahiru Peris	passenger1@streetify.com	+94771111111	PASSENGER	NULL	1	0	5000.00
9	9	PASSENGER	Gihan Devis	passenger2@streetify.com	+94772222222	PASSENGER	NULL	1	0	1500.00
10	10	PASSENGER	Kasun Fernando	kasun@streetify.com	+94773333333	PASSENGER	NULL	1	0	250.00
11	11	DRIVER	Kamal Perera	driver1@streetify.com	+94774444444	DRIVER	NULL	1	0	8500.00
12	12	DRIVER	Nimal Silva	driver2@streetify.com	+94775555555	DRIVER	NULL	1	0	3200.00
13	13	DRIVER	Sunil Shantha	driver3@streetify.com	+94776666666	DRIVER	NULL	1	0	0.00
14	14	USER	System Admin	admin@streetify.lk	0000000000	ADMIN	SUPER_ADMIN	1	0	0.00
15	15	DRIVER	henuka dissanayake	henuka@gmail.com	0743486086	DRIVER	NULL	1	0	0.00

```sql
-- Check suspended/banned accounts (Security Audit)
SELECT id, email, role, suspended, suspension_reason, suspended_until
FROM users WHERE suspended = 1;
```

72 % No issues found

Results Messages

id	email	role	suspended	suspension_reason	suspended_until

```sql
-- Passengers ranked by wallet balance (Credit Tier Analysis)
SELECT id AS passenger_id, first_name+' '+last_name AS passenger_name,
       wallet_balance, preferred_payment_method
FROM users WHERE dtype = 'PASSENGER'
ORDER BY wallet_balance DESC;
```

72 % No issues found

Results Messages

	passenger_id	passenger_name	wallet_balance	preferred_payment_method
1	8	Lahiru Peris	5000.00	WALLET
2	9	Gihan Devis	1500.00	CARD
3	10	Kasun Fernando	250.00	CASH

Chanuka (IT25102207) — Booking & Trip Management

```sql
-- Full trip dashboard: passenger + driver + status + fare
SELECT t.id, t.status, t.ride_type,
    p.first_name+' '+p.last_name AS passenger,
    ISNULL(d.first_name+' '+d.last_name, 'NOT_ASSIGNED') AS driver,
    t.pickup_address, t.dropoff_address, t.distance_km,
    t.total_fare, t.payment_method, t.is_paid, t.created_at
FROM trips t
INNER JOIN users p ON t.passenger_id = p.id
LEFT  JOIN users d ON t.driver_id    = d.id
ORDER BY t.id DESC;
```

72 % — No issues found

Results | Messages

	id	status	ride_type	passenger	driver	pickup_address	dropoff_address	distance_km	total_fare	payment_method	is_paid	created_at
1	4	REQUESTED	CAR	Lahiru Peris	NOT_ASSIGNED	Mount Lavinia Hotel	Galle Face Green	12	2150	CARD	0	2026-09-27 11:16:28.0633333
2	3	IN_PROGRESS	CAR	Kasun Fernando	Kamal Perera	Bambalapitiya	Dehiwala Zoo	5	1100	CASH	0	2026-09-27 11:01:28.0633333
3	2	COMPLETED	TUK	Gihan Devis	Nimal Silva	SLIIT Malabe	Kottawa Bus Stand	11.2	1076	WALLET	1	2026-09-27 10:16:28.0633333
4	1	COMPLETED	CAR	Lahiru Peris	Kamal Perera	Colombo Fort	Nugegoda Junction	10.5	1925	CARD	1	2026-09-27 08:16:28.0633333

```sql
-- Revenue breakdown by ride type (TUK, CAR, VAN, BIKE)
SELECT ride_type, COUNT(*) AS total_trips,
    AVG(distance_km) AS avg_distance_km,
    SUM(total_fare)  AS total_revenue
FROM trips GROUP BY ride_type;
```

72 % — No issues found

Results | Messages

	ride_type	total_trips	avg_distance_km	total_revenue
1	CAR	3	9.16666666666667	5175
2	TUK	1	11.2	1076

Tharindu (IT25102241) — Driver Management & Verification

```sql
-- Approved drivers with vehicle & GPS location
SELECT u.id, u.first_name+' '+u.last_name AS driver_name,
    u.license_number, u.verification_status, u.average_rating,
    u.is_online, u.current_lat, u.current_lng,
    v.vehicle_type, v.number_plate, v.make+' '+v.model AS vehicle
FROM users u
LEFT JOIN vehicles v ON v.driver_id = u.id
WHERE u.dtype = 'DRIVER' AND u.verification_status = 'APPROVED';
```

72 % — No issues found

Results | Messages

	id	driver_name	license_number	verification_status	average_rating	is_online	current_lat	current_lng	vehicle_type	number_plate	vehicle
1	11	Kamal Perera	B1234567	APPROVED	4.9	1	6.9271	79.8612	CAR	WP CAB-1234	Toyota Prius
2	12	Nimal Silva	B9876543	APPROVED	4.4	1	6.8649	79.8997	TUK	WP ABF-5678	Bajaj RE 4S
3	15	henuka dissanayake	NULL	APPROVED	0	0	NULL	NULL	moto	AAB-0934	NULL

```sql
-- Pending verification queue (document review backlog)
SELECT id, first_name+' '+last_name AS name, email,
       license_number, verification_status, created_at
FROM users
WHERE dtype = 'DRIVER' AND verification_status = 'PENDING_VERIFICATION';
```

72 % — No issues found

Results | Messages

	id	name	email	license_number	verification_status	created_at
1	13	Sunil Shantha	driver3@streetify.com	B5554321	PENDING_VERIFICATION	2026-09-27 11:14:32.5866667

```sql
-- Driver document review queue
SELECT d.id, u.first_name+' '+u.last_name AS driver_name,
    d.doc_type, d.file_path, d.status, d.reviewer_note, d.uploaded_at
FROM driver_documents d
INNER JOIN users u ON d.driver_id = u.id
ORDER BY d.uploaded_at DESC;
```

72 % — No issues found

Results | Messages

	id	driver_name	doc_type	file_path	status	reviewer_note	uploaded_at
1	7	Sunil Shantha	license	uploads/documents/driver-sunil/license.pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333
2	8	Sunil Shantha	reg	uploads/documents/driver-sunil/reg.pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333
3	9	Sunil Shantha	insurance	uploads/documents/driver-sunil/insurance.pdf	PENDING	Awaiting review by Driver Admin Tharindu	2026-09-27 10:50:25.2433333
4	4	Nimal Silva	license	uploads/documents/driver-nimal/license.jpg	APPROVED	Tuk-tuk commercial license verified (Tharindu)	2026-09-22 12:50:25.2433333
5	5	Nimal Silva	reg	uploads/documents/driver-nimal/reg.pdf	APPROVED	Western Province revenue license confirmed	2026-09-22 12:50:25.2433333
6	6	Nimal Silva	insurance	uploads/documents/driver-nimal/insurance.pdf	APPROVED	Third-party commercial insurance verified	2026-09-22 12:50:25.2433333
7	1	Kamal Perera	license	uploads/documents/driver-kamal/license.pdf	APPROVED	Verified by Tharindu (Driver Admin) - Valid until 2029	2026-09-17 12:50:25.2433333
8	2	Kamal Perera	reg	uploads/documents/driver-kamal/reg.pdf	APPROVED	Revenue license valid until 2027 (Tharindu)	2026-09-17 12:50:25.2433333
9	3	Kamal Perera	insurance	uploads/documents/driver-kamal/insurance.pdf	APPROVED	Comprehensive commercial insurance policy verified	2026-09-17 12:50:25.2433333

Daham (IT25102225) — Payment & Financial Settlement

```sql
-- Full payment ledger with 15%/85% commission split
SELECT p.id, p.trip_id,
    pass.first_name+' '+pass.last_name AS passenger,
    drv.first_name+' '+drv.last_name  AS driver,
    p.gross_amount, p.platform_commission AS commission_15pct,
    p.driver_net AS driver_85pct, p.payment_method,
    p.status, p.transaction_ref, p.processed_at
FROM payments p
INNER JOIN users pass ON p.passenger_id = pass.id
LEFT  JOIN users drv  ON p.driver_id    = drv.id
ORDER BY p.id DESC;
```

72 % — No issues found

Results | Messages

	id	trip_id	passenger	driver	gross_amount	commission_15pct	driver_85pct	payment_method	status	transaction_ref	processed_at
1	2	2	Gihan Devis	Nimal Silva	1076	161.4	914.6	WALLET	SUCCESS	TXN_WLT_110294	2026-09-27 11:18:52.0833333
2	1	1	Lahiru Peris	Kamal Perera	1925	288.75	1636.25	CARD	SUCCESS	TXN_CARD_894312	2026-09-27 09:18:52.0833333

```sql
-- Total platform revenue vs total driver payouts (executive KPI)
SELECT COUNT(*) AS total_payments,
    SUM(gross_amount)          AS total_fares,
    SUM(platform_commission)   AS streetify_revenue,
    SUM(driver_net)            AS driver_payouts
FROM payments WHERE status = 'SUCCESS';
```

72 % — No issues found

Results | Messages

	total_payments	total_fares	streetify_revenue	driver_payouts
1	2	3001	450.15	2550.85

```sql
-- Breakdown by payment channel
SELECT payment_method, COUNT(*) AS transactions,
    SUM(gross_amount) AS total_amount
FROM payments GROUP BY payment_method;
```

72 % — No issues found

Results | Messages

	payment_method	transactions	total_amount
1	CARD	1	1925
2	WALLET	1	1076

Mithun (IT25102193) — Review & Dispute Management

```sql
-- All reviews with passenger and driver names
SELECT r.id, r.trip_id,
    p.first_name+' '+p.last_name AS passenger,
    d.first_name+' '+d.last_name AS driver,
    r.rating, r.comment, r.created_at
FROM reviews r
INNER JOIN users p ON r.passenger_id = p.id
INNER JOIN users d ON r.driver_id    = d.id
ORDER BY r.id DESC;
```

72 % No issues found

Results Messages

	id	trip_id	passenger	driver	rating	comment	created_at
1	2	2	Gihan Devis	Nimal Silva	4	Quick tuk ride, navigated through Kottawa traff...	2026-09-27 11:18:52.1033333
2	1	1	Lahiru Peris	Kamal Perera	5	Super clean car, friendly driver and arrived rig...	2026-09-27 09:18:52.1033333

```sql
-- Driver performance tier classification
SELECT id, first_name+' '+last_name AS driver_name,
    average_rating, total_trips,
    CASE WHEN average_rating >= 4.8 THEN 'Top Rated'
         WHEN average_rating >= 4.0 THEN 'Good'
         ELSE 'Needs Attention' END AS tier
FROM users WHERE dtype = 'DRIVER' ORDER BY average_rating DESC;
```

72 % No issues found

Results Messages

	id	driver_name	average_rating	total_trips	tier
1	13	Sunil Shantha	5	0	Top Rated
2	11	Kamal Perera	4.9	142	Top Rated
3	12	Nimal Silva	4.4	48	Good
4	15	henuka dissanayake	0	0	Needs Attention

```sql
-- Dispute resolution status + refund summary
SELECT dispute_type, status, COUNT(*) AS count,
    SUM(approved_refund_amount) AS total_refunded
FROM dispute_tickets GROUP BY dispute_type, status;
```

72 % No issues found

Results Messages

	dispute_type	status	count	total_refunded
1	OVERCHARGED	RESOLVED	1	100

Vidura (IT25102240) — Super Admin & Governance

```sql
-- Platform executive summary: users by role
SELECT role, COUNT(*) AS total,
    SUM(CASE WHEN active=1    THEN 1 ELSE 0 END) AS active_count,
    SUM(CASE WHEN suspended=1 THEN 1 ELSE 0 END) AS suspended_count
FROM users GROUP BY role;
```

72 % No issues found

Results Messages

	role	total	active_count	suspended_count
1	ADMIN	8	8	0
2	DRIVER	4	4	0
3	PASSENGER	3	3	0

```sql
-- Trip pipeline overview (all statuses with volume)
SELECT status, COUNT(*) AS total_trips,
    ISNULL(SUM(total_fare), 0) AS volume_lkr
FROM trips GROUP BY status;
```

72 % No issues found

Results Messages

	status	total_trips	volume_lkr
1	COMPLETED	2	3001
2	IN_PROGRESS	1	1100
3	REQUESTED	1	2150

```sql
-- Today's earnings dashboard (real-time KPI)
SELECT CAST(GETDATE() AS DATE) AS report_date,
    COUNT(*)                          AS completed_today,
    ISNULL(SUM(total_fare), 0)     AS gross_fares,
    ISNULL(SUM(platform_commission),0) AS net_commission
FROM trips
WHERE status = 'COMPLETED'
  AND CAST(completed_at AS DATE) = CAST(GETDATE() AS DATE);
```

72 % No issues found

Results Messages

	report_date	completed_today	gross_fares	net_commission
1	2026-09-27	2	3001	450.15

```sql
-- Full audit trail (last 20 admin actions)
SELECT TOP 20 id, performed_by_email, action_type,
    description, target_entity_type, target_entity_id, created_at
FROM audit_logs ORDER BY created_at DESC;
```

72 % No issues found

Results Messages

	id	performed_by_email	action_type	description	target_entity_type	target_entity_id	created_at
1	2	admin@streetify.com	DISPUTE_RESOLVED	Approved partial refund of 100 LKR.	DISPUTE_TICKET	1	2026-09-27 11:18:52.1166667
2	1	admin@streetify.com	ADMIN_LOGIN	Super Admin logged in.	USER	1	2026-09-27 11:18:52.1166667

Part E: Stored Procedure — usp_ProcessTripPayment

This stored procedure encapsulates the complete payment finalization workflow for Streetify. It enforces ACID compliance through explicit transaction management, validates business rules before any mutation, and handles driver commission debt tracking for cash payments. It is the single authoritative entry point for all payment processing.

E.1 Business Logic & Design Rationale

Component	Design Decision	Justification
Input Validation Guard 1	Checks trip EXISTS and status = 'COMPLETED'	Prevents payments for in-progress or cancelled trips at the DB level
Input Validation Guard 2	Checks no payment row exists for trip_id	Guarantees idempotency — safe for network retries without double billing
15% Commission Rule	Deducts 15% as platform_commission, 85% to driver_net	Core Streetify business model — immutably recorded at time of payment
CASH commission_debt	Adds 15% to drivers.commission_debt for cash trips	Driver received cash physically; platform records what is owed separately
WALLET deduction	Deducts gross_amount from passenger wallet_balance	Atomic operation — cannot leave wallet in partial state
Transaction Reference	Generates unique TXN_METHOD_random ID	Enables external payment gateway reconciliation and audit tracing
SET XACT_ABORT ON	Auto-rollback entire TX on any T-SQL error	Prevents partial data states without manual ROLLBACK in every error branch
TRY...CATCH + XACT_STATE()	Catches all errors, checks TX state before ROLLBACK	XACT_STATE() = -1 means TX is uncommittable; 0 means no TX active — handle both

E.2 Complete Stored Procedure Code

usp_ProcessTripPayment — ACID-Compliant Payment Procedure

```
USE streetify_db;
GO

SET QUOTED_IDENTIFIER ON;
GO

CREATE OR ALTER PROCEDURE usp_ProcessTripPayment
    @TripID      BIGINT,
    @Method      NVARCHAR(20),    -- Accepted: 'CARD' | 'WALLET' | 'CASH'
    @AmountPaid  DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;   -- Auto-rollback entire TX on any statement-level error

    -- ── Guard 1: Trip must exist and be in COMPLETED state ──────────────
    IF NOT EXISTS (
        SELECT 1 FROM trips WHERE id = @TripID AND status = 'COMPLETED'
    )
    BEGIN
        ;THROW 50001, 'ERROR: Trip does not exist or is not in COMPLETED state.', 1;
        RETURN;
    END

    -- ── Guard 2: Idempotency check — no duplicate payment allowed ────────
    IF EXISTS (SELECT 1 FROM payments WHERE trip_id = @TripID)
    BEGIN
        ;THROW 50002, 'ERROR: Payment already processed for this trip.', 1;
        RETURN;
    END

    -- ── Variable Declarations ────────────────────────────────────────────
    DECLARE @DriverID    BIGINT         = (SELECT driver_id    FROM trips WHERE id = @TripID);
    DECLARE @PassengerID BIGINT         = (SELECT passenger_id FROM trips WHERE id = @TripID);
    DECLARE @Commission  DECIMAL(10,2)  = @AmountPaid * 0.15;   -- 15% platform cut
    DECLARE @NetShare    DECIMAL(10,2)  = @AmountPaid - @Commission;  -- 85% to driver
    DECLARE @TxnRef      NVARCHAR(100)  =
        'TXN_' + @Method + '_' + CAST(ABS(CHECKSUM(NEWID())) AS NVARCHAR(20));
```

```
BEGIN TRY
    BEGIN TRANSACTION;

        -- Step 1: Insert immutable payment record
        INSERT INTO payments (trip_id, passenger_id, driver_id, gross_amount,
            platform_commission, driver_net, payment_method, status,
            transaction_ref, processed_at)
        VALUES (@TripID, @PassengerID, @DriverID, @AmountPaid,
            @Commission, @NetShare, @Method, 'SUCCESS', @TxnRef, GETDATE());

        -- Step 2: Mark trip as paid
        UPDATE trips
        SET is_paid    = 1,
            updated_at = GETDATE()
        WHERE id = @TripID;

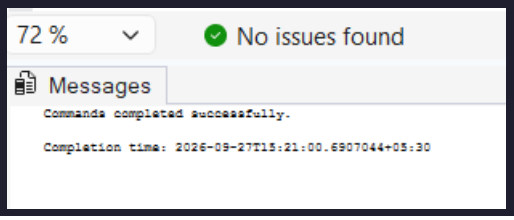
        -- Step 3: CASH — driver received physical cash, record commission debt
        IF @Method = 'CASH'
        BEGIN
            UPDATE users
            SET commission_debt = commission_debt + @Commission,
                updated_at      = GETDATE()
            WHERE id = @DriverID;
        END

        -- Step 4: WALLET — deduct fare from passenger's digital wallet
        IF @Method = 'WALLET'
        BEGIN
            UPDATE users
            SET wallet_balance = wallet_balance - @AmountPaid,
                updated_at     = GETDATE()
            WHERE id = @PassengerID;
        END

    COMMIT TRANSACTION;
```

```
        -- Return payment receipt to the caller (application layer)
        SELECT
            @TripID     AS trip_id,
            @AmountPaid AS gross_paid,
            @Commission AS platform_commission_15pct,
            @NetShare   AS driver_net_85pct,
            @Method     AS payment_method,
            @TxnRef     AS transaction_reference,
            GETDATE()   AS processed_at;

    END TRY
    BEGIN CATCH
        -- XACT_STATE() = -1: uncommittable TX; <> 0: any active TX
        IF XACT_STATE() <> 0
            ROLLBACK TRANSACTION;
        ;THROW;  -- Re-raise original error to calling application
    END CATCH
END;
GO
```



Inline SQL Code (complete procedure — also visible as screenshot below):

```
-- ═══════════════════════════════════════════════════════════════════════
--  STREETIFY — STORED PROCEDURE: usp_ProcessTripPayment (ACID Transaction)
-- ═══════════════════════════════════════════════════════════════════════

USE streetify_db;
GO

SET QUOTED_IDENTIFIER ON;
GO

CREATE OR ALTER PROCEDURE usp_ProcessTripPayment
    @TripID      BIGINT,
    @Method      NVARCHAR(20),    -- Accepted: 'CARD' | 'WALLET' | 'CASH'
    @AmountPaid  DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;   -- Auto-rollback entire TX on any statement-level error

    -- ── Guard 1: Trip must exist and be in COMPLETED state ──────────────
    IF NOT EXISTS (
        SELECT 1 FROM trips WHERE id = @TripID AND status = 'COMPLETED'
    )
    BEGIN
        ;THROW 50001, 'ERROR: Trip does not exist or is not in COMPLETED state.', 1;
        RETURN;
    END

    -- ── Guard 2: Idempotency check — no duplicate payment allowed ──────
    IF EXISTS (SELECT 1 FROM payments WHERE trip_id = @TripID)
    BEGIN
        ;THROW 50002, 'ERROR: Payment already processed for this trip.', 1;
        RETURN;
    END

    -- ── Variable Declarations ──────────────────────────────────────────
    DECLARE @DriverID    BIGINT         = (SELECT driver_id    FROM trips WHERE id = @TripID);
    DECLARE @PassengerID BIGINT         = (SELECT passenger_id FROM trips WHERE id = @TripID);
    DECLARE @Commission  DECIMAL(10,2)  = @AmountPaid * 0.15;   -- 15% platform cut
    DECLARE @NetShare    DECIMAL(10,2)  = @AmountPaid - @Commission;  -- 85% to driver
    DECLARE @TxnRef      NVARCHAR(100)  =
        'TXN_' + @Method + '_' + CAST(ABS(CHECKSUM(NEWID())) AS NVARCHAR(20));

    BEGIN TRY
        BEGIN TRANSACTION;

            -- Step 1: Insert immutable payment record
            INSERT INTO payments (trip_id, passenger_id, driver_id, gross_amount,
                platform_commission, driver_net, payment_method, status,
                transaction_ref, processed_at)
            VALUES (@TripID, @PassengerID, @DriverID, @AmountPaid,
                @Commission, @NetShare, @Method, 'SUCCESS', @TxnRef, GETDATE());

            -- Step 2: Mark trip as paid
            UPDATE trips
```

```
            SET is_paid    = 1,
                updated_at = GETDATE()
            WHERE id = @TripID;


            -- Step 3: CASH — driver received physical cash, record commission debt
            IF @Method = 'CASH'
            BEGIN
                UPDATE users
                SET commission_debt = commission_debt + @Commission,
                    updated_at      = GETDATE()
                WHERE id = @DriverID;
            END


            -- Step 4: WALLET — deduct fare from passenger's digital wallet
            IF @Method = 'WALLET'
            BEGIN
                UPDATE users
                SET wallet_balance = wallet_balance - @AmountPaid,
                    updated_at     = GETDATE()
                WHERE id = @PassengerID;
            END


        COMMIT TRANSACTION;


        -- Return payment receipt to the caller (application layer)
        SELECT
            @TripID                   AS trip_id,
            @AmountPaid               AS gross_paid,
            @Commission               AS platform_commission_15pct,
            @NetShare                 AS driver_net_85pct,
            @Method                   AS payment_method,
            @TxnRef                   AS transaction_reference,
            CONVERT(VARCHAR(19), GETDATE(), 120) AS processed_at;


    END TRY
    BEGIN CATCH
        -- XACT_STATE() = -1: uncommittable TX; <> 0: any active TX
        IF XACT_STATE() <> 0
            ROLLBACK TRANSACTION;
        ;THROW;  -- Re-raise original error to calling application
    END CATCH
END;
GO


PRINT '=================================================================';
PRINT ' Stored Procedure usp_ProcessTripPayment Created Successfully!';
PRINT '=================================================================';
GO
```

E.3 Demonstration Script

E.3 — Demo Execution Script

```
USE streetify_db;
GO

-- ── DEMO: Process a CASH payment for Trip 3 ──────────────────────
-- First: Mark Trip 3 as COMPLETED (it is IN_PROGRESS in seeded data)
UPDATE trips SET status = 'COMPLETED', completed_at = GETDATE() WHERE id = 3;
GO

-- Execute the procedure with CASH method (Bambalapitiya -> Dehiwala, LKR 1100)
EXEC usp_ProcessTripPayment
    @TripID     = 3,
    @Method     = 'CASH',
    @AmountPaid = 1100.00;
GO
```

95 % — No issues found

Messages

```
(1 row affected)
Msg 50002, Level 16, State 1, Procedure usp_ProcessTripPayment, Line 23 [Batch Start Line 7]
ERROR: Payment already processed for this trip.

Completion time: 2026-09-27T16:45:18.5854523+05:30
```

```
-- Verify 1: Driver's commission_debt increased by LKR 165 (15% of 1100)
SELECT id, first_name+' '+last_name AS driver_name,
       commission_debt, wallet_balance
FROM users WHERE email = 'driver1@streetify.com';
```

87 % — No issues found

Results | Messages

	id	driver_name	commission_debt	wallet_balance
1	11	Kamal Perera	165.00	8500.00

```
-- Verify 2: Payment record was created with correct split
SELECT trip_id, gross_amount, platform_commission, driver_net,
       payment_method, status, transaction_ref
FROM payments WHERE trip_id = 3;
```

79 % — No issues found

Results | Messages

	trip_id	gross_amount	platform_commission	driver_net	payment_method	status	transaction_ref
1	3	1100	165	935	CASH	SUCCESS	TXN_CASH_2106163413

```sql
-- Verify 3: trips.is_paid flag updated
SELECT id, status, is_paid FROM trips WHERE id = 3;
```

79 % No issues found

Results Messages

	id	status	is_paid
1	3	COMPLETED	1

```sql
-- Test Guard: Try to pay Trip 3 AGAIN (should throw error 50002)
BEGIN TRY
    EXEC usp_ProcessTripPayment @TripID=3, @Method='CARD', @AmountPaid=1100.00;
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS error_no, ERROR_MESSAGE() AS error_msg;
END CATCH;
```

79 % No issues found

Results Messages

	error_no	error_msg
1	50002	ERROR: Payment already processed for this trip.

Part F: Trigger — trg_ProtectFinalizedTrips

This AFTER UPDATE trigger is the final safety gate protecting financial record integrity. Once a trip reaches a terminal state (COMPLETED or CANCELLED), its financial data (total_fare, platform_commission, driver_net, distance_km, status) becomes immutable — protecting the platform from both accidental corruption and intentional fraud.

F.1 Trigger Design Rationale

Design Aspect	Explanation
AFTER UPDATE (not INSTEAD OF)	An AFTER trigger fires after changes are staged but before commit. It can inspect both old and new state and issue ROLLBACK to undo the change. INSTEAD OF would require reimplementing the update logic manually — more complex and error-prone.
Virtual tables: inserted and deleted	SQL Server auto-populates 'deleted' (before-state) and 'inserted' (after-state) for every DML operation. We JOIN them on id to compare old vs new field values.
TRIGGER_NESTLEVEL() guard	Prevents infinite recursion if this trigger fires another trigger. If nest level > 1, we return immediately to avoid stack overflow.
Protected columns	total_fare, platform_commission, driver_net, distance_km, status. These are the fields that constitute the immutable financial record. Administrative notes and timestamps are not protected.
THROW 50101 + ROLLBACK pattern	THROW raises a descriptive error with number 50101. ROLLBACK undoes the UPDATE. The calling application (Spring Boot @Transactional) catches the exception and surfaces it as an API error response.

F.2 Complete Trigger Code

trg_ProtectFinalizedTrips — Immutable Financial Record Guard

Inline SQL Code (complete trigger — also visible as screenshot below):

```
-- ============================================================================
--  STREETIFY — TRIGGER: trg_ProtectFinalizedTrips (Immutable Audit Guard)
-- ============================================================================

USE streetify_db;
GO

SET QUOTED_IDENTIFIER ON;
GO

CREATE OR ALTER TRIGGER trg_ProtectFinalizedTrips
ON trips
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Guard: prevent recursive trigger execution
    IF TRIGGER_NESTLEVEL(@@PROCID) > 1 RETURN;

    -- Check: Was the ORIGINAL status a terminal state (COMPLETED or CANCELLED)?
    --        AND: Is someone trying to mutate protected financial columns?
    IF EXISTS (
        SELECT 1
        FROM deleted d
        INNER JOIN inserted i ON d.id = i.id
        WHERE d.status IN ('COMPLETED', 'CANCELLED')
          AND (
                d.total_fare           <> i.total_fare
             OR d.platform_commission  <> i.platform_commission
             OR d.driver_net           <> i.driver_net
             OR d.distance_km          <> i.distance_km
             OR d.status               <> i.status
          )
    )
    BEGIN
        ROLLBACK TRANSACTION;
        ;THROW 50101,
            'CRITICAL INTEGRITY VIOLATION: Finalized trips (COMPLETED/CANCELLED) are immutable
financial records. Fare, commission, distance, and status cannot be altered after
finalization.',
            1;
    END
END;
GO
```

```sql
USE streetify_db;
GO

SET QUOTED_IDENTIFIER ON;
GO

CREATE OR ALTER TRIGGER trg_ProtectFinalizedTrips
ON trips
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Guard: prevent recursive trigger execution
    IF TRIGGER_NESTLEVEL(@@PROCID) > 1 RETURN;

    -- Check: Was the ORIGINAL status a terminal state (COMPLETED or CANCELLED)?
    --        AND: Is someone trying to mutate protected financial columns?
    IF EXISTS (
        SELECT 1
        FROM deleted d
        INNER JOIN inserted i ON d.id = i.id
        WHERE d.status IN ('COMPLETED', 'CANCELLED')
          AND (
                d.total_fare           <> i.total_fare
            OR d.platform_commission   <> i.platform_commission
            OR d.driver_net            <> i.driver_net
            OR d.distance_km           <> i.distance_km
            OR d.status                <> i.status
          )
    )
    BEGIN
        ;THROW 50101,
            'CRITICAL INTEGRITY VIOLATION: Finalized trips (COMPLETED/CANCELLED) are immutable financial records.
            Fare, commission, distance, and status cannot be altered after finalization.',
            1;

        ROLLBACK TRANSACTION;
    END
END;
GO
```

9 % No issues found

Messages

Commands completed successfully.

F.3 Demonstration Script

F.3 —Testing Trigger Protection

```
-- ── Attempt to tamper with a COMPLETED trip ──────────────────────
-- Trip 1 (Colombo Fort -> Nugegoda) is COMPLETED with total_fare = 1925.00

-- Test 1: Attempt to change the fare (SHOULD FAIL with error 50101)
BEGIN TRY
    UPDATE trips SET total_fare = 1.00 WHERE id = 1;   -- Fraudulent edit!
    PRINT 'Update succeeded (UNEXPECTED — trigger failed!)';
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER()   AS error_number,
        ERROR_MESSAGE()  AS error_message,
        ERROR_SEVERITY() AS severity;
END CATCH;
GO
```

79 % — No issues found

Results | Messages

	error_number	error_message	severity
1	50101	CRITICAL INTEGRITY VIOLATION: Finalized trips (CO...	16

```
-- Test 2: Attempt to reopen a COMPLETED trip (SHOULD FAIL)
BEGIN TRY
    UPDATE trips SET status = 'REQUESTED' WHERE id = 1;
END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE() AS error_message;
END CATCH;
GO
```

79 % — No issues found

Results | Messages

	error_message
1	CRITICAL INTEGRITY VIOLATION: Finalized trips (CO...

```
-- Test 3: Verify Trip 1 data is completely UNCHANGED after both attempts
SELECT id, pickup_address, dropoff_address, total_fare,
       platform_commission, driver_net, status
FROM trips WHERE id = 1;
GO
```

79 % — No issues found

Results | Messages

	id	pickup_address	dropoff_address	total_fare	platform_commission	driver_net	status
1	1	Colombo Fort	Nugegoda Junction	1925	288.75	1636.25	COMPLETED

```
-- Test 4: Verify a NON-financial update STILL works (trigger does not over-block)
UPDATE trips SET updated_at = GETDATE() WHERE id = 1;
SELECT id, updated_at FROM trips WHERE id = 1;
GO
```

79 % — No issues found

Results | Messages

	id	updated_at
1	1	2026-09-27 17:23:11.6266667

References

[1] Elmasri, R. & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson. [ER-to-Relational Mapping Rules — Chapter 9; Normalization — Chapter 15]

[2] Microsoft Corporation. (2024). Transact-SQL Reference (T-SQL): CREATE TABLE, CREATE INDEX, CREATE PROCEDURE, CREATE TRIGGER, XACT_ABORT, TRY...CATCH, THROW. https://learn.microsoft.com/en-us/sql/t-sql/

[3] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.

[4] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley. [Single Table Inheritance Pattern — pp. 278-284]

[5] Hibernate ORM Documentation. (2024). Single Table Inheritance (STI) — JPA / Hibernate. https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html

[6] Streetify Group 24. (2026). Assignment 01 — Enhanced Entity-Relationship (EER) Diagram. docs/2026-Y2-S1-KU-24 (Group 24)_Assignment01_EER.pdf.

[7] Streetify Group 24. (2026). Assignment 01 — System Requirements Specification. docs/2026-Y2-S1-KU-24 (Group 24)_Assignment01_Requirements.pdf.

[8] Streetify Group 24. (2026). Project Proposal Report. docs/2026-Y2-S1-KU-24_Proposal_Report.pdf.

[9] Streetify Group 24. (2026). Source Code & Database Scripts. database/01_schema_ddl.sql, database/02_seed_data.sql, database/03_team_member_queries.sql, database/04_utility_test_scripts.sql.

[10] Streetify Group 24. (2026). Project Evaluation Guide. Streetify_Evaluation_Guide.md.